GitHub repository: https://github.com/Mmukund-31/RouteProof

Live application: NOT DEPLOYED - no live frontend exists yet | **Backend health:** NOT DEPLOYED - no live backend exists yet

RouteProof

Technical Design Document — Proof of Delivery Intelligence Platform

1. Project overview

RouteProof turns a physical delivery document into a verified, traceable and billing-ready digital record for a full-truckload transporter (Chhatrapati Sambhajinagar; ~2,200 trips/month). Flow: **trip assigned > driver captures POD (offline-capable PWA) > upload > vision model extracts structured fields > deterministic validation > human review of exceptions > verified POD > billing queue > Tally-compatible CSV**. The original photograph stays the source of truth; the AI never authorises an invoice and the system never issues, sends or posts one. Two roles exist: *driver* (own trips only) and *billing executive* (workspace). Eight primary screens: (1) operations dashboard, (2) trips, (3) driver trip list, (4) POD capture, (5) result / retake, (6) review queue and review screen, (7) billing (queue, export history, client links), (8) POD viewer (staff review screen and the read-only client view link). Login, the hired-driver upload page, the full missing-POD list and audit history are supporting views.

2. GitHub and live deployment links

GitHub: **https://github.com/Mmukund-31/RouteProof**. Live frontend: **NOT DEPLOYED - no live frontend exists yet**. Health endpoint: **NOT DEPLOYED - no live backend exists yet**. The source is published at the GitHub URL above. The Dockerfile, `render.yaml` and step-by-step instructions are complete, but the application has **not** been deployed: no live frontend or backend exists yet and no deployment claim is made.

3. Client problem and product objectives

Signed lorry receipts return 10–20 days after delivery; WhatsApp photos are lost; invoices cannot be raised without POD, so working capital is locked. Client-provided figures (**assumptions supplied by the client, not verified by us**): POD-to-invoice 18 days today, 4-day target; ~Rs 3.24 crore working capital released; ~Rs 38.9 lakh annual financing benefit; ~Rs 63.36 lakh/yr lost to lost or disputed PODs; ~Rs 1.02 crore total opportunity. This build instruments the pipeline (upload latency, upload-to-billing-ready time) so that such figures can later be measured; it does **not** assert any achieved improvement.

- Capture POD at the consignee with one obvious action, even with no network.
- Extract LR, consignee, quantity, date, signature/stamp visibility and shortage/damage remarks with a vision model; never trust confidence alone.
- Route every doubtful case to a person with the original image beside the AI values; preserve AI originals and every attempt.
- Make only verified trips billable; export a documented CSV; keep a complete audit trail.

4. Explicit assumptions

Assumption A1. AI confidence threshold 0.85 on critical fields (LR, consignee, delivered quantity, date) is an implementation default, **not** a calibrated probability; it is configurable (`AI_CONFIDENCE_THRESHOLD`) and recorded with every extraction.

Assumption A2. When the model returns output and *all* deterministic checks pass (LR match, consignee match, plausible date, no shortage/damage, stamp and signature visible, confidences above threshold, quantity equal if an expected quantity exists), the POD is auto-verified (*verification_method* = *ai*) and the trip becomes billing-ready. A reviewer may prefer a mandatory human step; that is a one-line policy change.

Assumption A3. Consignee match = case/punctuation-insensitive substring or >= 0.70 sequence similarity. Date plausible = not in the future and within 7 days before to 30 days after the trip delivery date. LR compared after stripping non-alphanumerics.

Assumption A4. Trips carry an optional expected quantity + unit (added to the specified schema). Quantity is compared *only* when it exists; otherwise the check is reported as not compared.

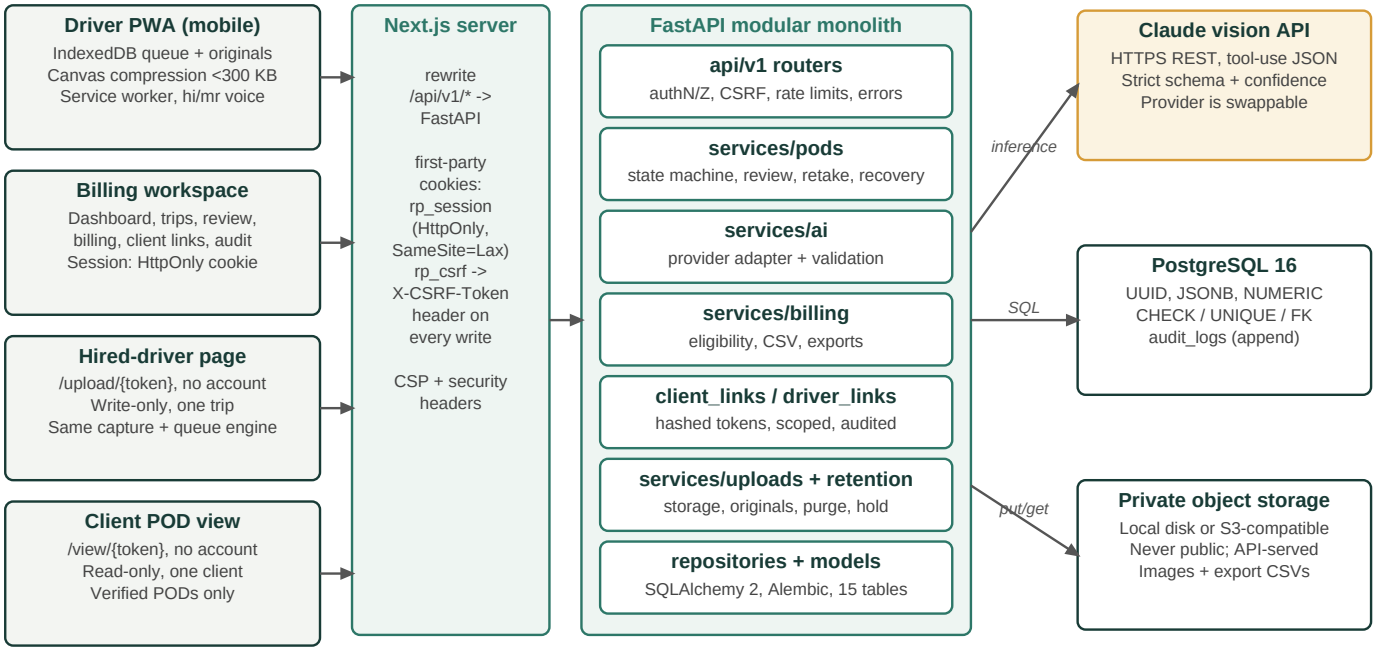
Assumption A5. Missing stamp, missing signature, missing LR or an unreadable image request a retake (max 2); a mismatch, shortage, damage, quantity/date problem or low confidence goes to review. After 2 retakes everything goes to review.

Assumption A6. Two-stage upload: the compressed (<300 KB) JPEG is sent first and processed by AI; once the server acknowledges it, the full-resolution original is attached separately (POST /pods/{id}/original, or the link equivalent) when the connection allows. The original stays in IndexedDB until that second upload is acknowledged; a failed original upload never removes the already-accepted compressed evidence or the copy on the phone.

Assumption A7. Client clocks and GPS are labelled as device-reported and are not proof; server receipt time is stored separately. Billing cycles are half-months (e.g. 2026-11-H1). A POD is 'overdue' after 3 days (configurable).

Assumption A8. Browser sessions are a signed JWT in an **HttpOnly, SameSite=Lax cookie** (unreadable by page JavaScript) with a session-bound double-submit CSRF token (rp_csrf cookie echoed as X-CSRF-Token on every write). Sessions expire after **60 minutes idle** (sliding refresh) and **12 hours absolute** from login. Logout clears the cookies and is audited, but a copied token stays valid until it expires (no server-side denylist). Rate limiting is an in-memory per-instance sliding window. All seeded data and demo credentials are fictional and non-production.

5. Architecture and technology decisions



Decision	Choice and reason
Modular monolith	One deployable FastAPI service with strict module seams (api / services / repositories / adapters); microservices would add cost with no benefit at ~2,200 trips/month.
Backend	Python 3.12+, FastAPI, Pydantic v2 (strict extraction schema, request validation), SQLAlchemy 2, Alembic, Argon2 (passwords), PyJWT in an HttpOnly cookie + CSRF; the Next.js server proxies /api/v1 so cookies stay first-party.
Frontend	Next.js 16 App Router, React 19, strict TypeScript, Tailwind v4 with shadcn-style components (button, input, select, badge, card, table, dialog, dropdown, toast, empty/error/skeleton states), Lucide, Recharts.
Persistence	PostgreSQL (JSONB, UUID, NUMERIC money, CHECK/UNIQUE/FK). SQLite is used only for fast unit tests via portable column types. All timestamps are UTC; the UI renders IST.
AI provider	`AIProvider.analyze_pod()` interface; `ClaudeVisionProvider` calls the Anthropic Messages API directly over HTTPS with tool-use JSON, validated against a strict schema (the Python SDK was avoided because a native dependency was blocked by Windows Application Control during development); `manual` fallback; deterministic `mock` for tests only.
Storage	Adapter: local disk (dev / Render persistent disk) or private S3-compatible bucket. Images and CSVs are never public; they are streamed through authenticated endpoints with *Cache-Control: private, no-store*.

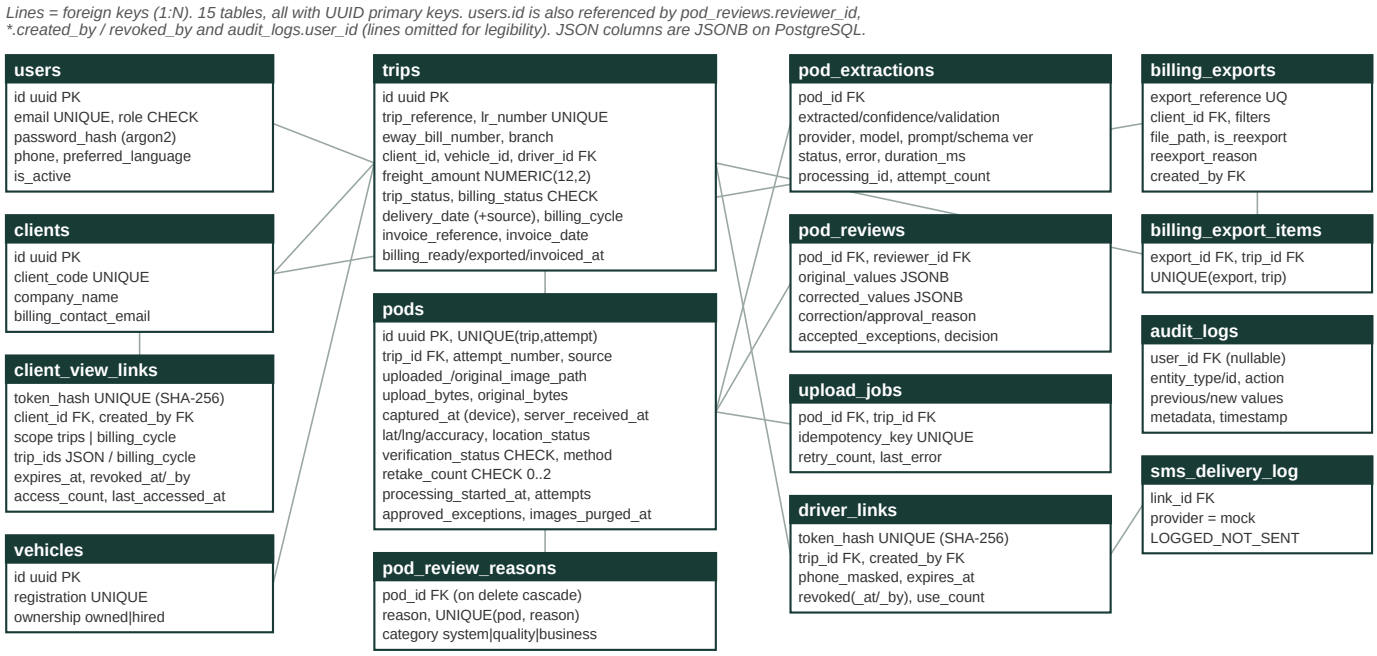
API surface (versioned under /api/v1)

Area	Endpoints (role)
Auth	POST /auth/login (rate-limited; sets HttpOnly session + CSRF cookies), GET /auth/me, POST /auth/logout
Trips	GET /trips (billing, search + filters + whitelisted sort + SQL pagination), GET /trips/{id} (billing; driver if assigned), POST /trips/import (billing, ?dry_run), GET /trips/{id}/history (billing), GET /meta/options

Area	Endpoints (role)
PODs	POST /trips/{id}/pods (Idempotency-Key; driver own / billing), POST /pods/{id}/original (full-resolution original), GET /pods/{id}, GET /pods/{id}/image?variant=upload|original (authenticated stream; 410 after purge), POST /pods/{id}/process, POST /pods/{id}/retake, POST /pods/maintenance/recover (billing)
Review	GET /reviews (status, reason, category, priority, client, search; filtered, ordered and paginated in SQL via pod_review_reasons), POST /pods/{id}/review (billing)
Driver	GET /driver/trips, GET /driver/uploads, POST /driver/uploads/sync (acknowledges idempotency keys)
Hired driver	POST /driver-links, GET /driver-links?trip_id=, POST /driver-links/{id}/revoke (billing); GET /driver-links/{token}, POST /driver-links/{token}/pods, POST /driver-links/{token}/pods/{pod_id}/original (token-scoped, no account)
Billing	GET /billing/queue, POST /billing/exports, GET /billing/exports, GET /billing/exports/{id}/download, POST /billing/trips/{id}/invoiced (invoice_reference + invoice_date, recorded manually)
Client links (staff)	POST /client-links (client + trip list or billing cycle, expiry 1-30 days), GET /client-links, POST /client-links/{id}/revoke (billing)
Client view (public)	GET /client-view/{token} (scoped trip list, verified PODs only), GET /client-view/{token}/trips/{trip_id}/pod-image; 410 when expired or revoked; every access audited
Dashboard / audit	GET /dashboard/metrics | pod-aging | client-breakdown | missing-pods | recent-activity, GET /audit-logs (billing). GET /health is unauthenticated.

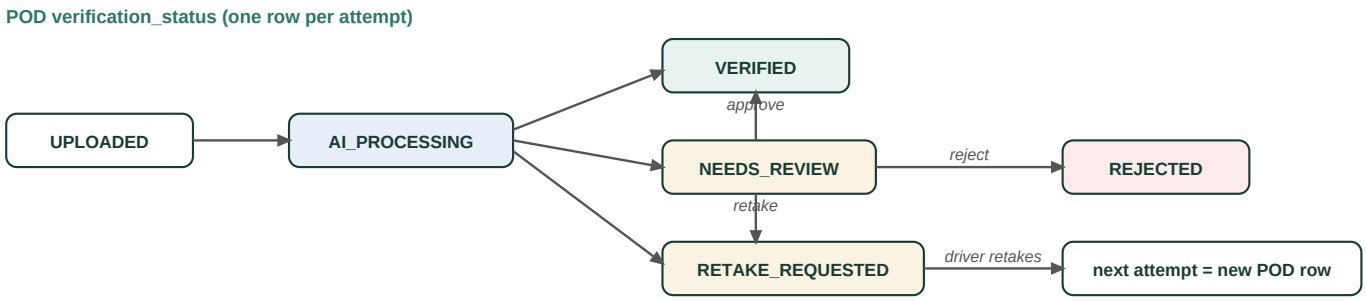
Errors use one envelope, `{error:{code,message,details}}`, with 400/401/403/404/409/413/415/422/429 used consistently. Request bodies are Pydantic models; responses are explicit serializers that never expose storage paths or token hashes.

6. Database schema and ER diagram



Fifteen tables, matching backend/app/models/entities.py: users, clients, vehicles, trips, pods, **pod_review_reasons** (one row per POD and reason, so the review queue filters in SQL), pod_extractions, pod_reviews, upload_jobs, **driver_links** (hashed single-trip upload tokens), **client_view_links** (hashed read-only client tokens scoped to trips or a billing cycle), **sms_delivery_log** (mock provider log), billing_exports, **billing_export_items** and audit_logs. Added columns beyond the brief: trips.consignee_name, expected_quantity, quantity_unit, billing_cycle, invoice_reference; pods.verification_method, verified_at, review_reasons, review_priority, source. Constraints: unique LR/trip reference, unique (trip, attempt), unique idempotency key, retake_count 0..2, status CHECKs, NUMERIC(12,2) freight. Evidence is never overwritten: each retake is a new pods row, each AI run a new pod_extractions row, each human decision a pod_reviews row. The Alembic baseline revision materialises this schema; later changes should be incremental revisions.

7. Main workflows



Trip status: IN_TRANSIT > DELIVERED > POD_PENDING > POD_VERIFIED (or CANCELLED). **Billing status:** NOT_READY > BILLING_READY > EXPORTED > INVOICED (recorded manually; invoices are issued outside RouteProof). **Retake cap:** two requests, after which the next attempt is routed to NEEDS_REVIEW. **Reprocess:** NEEDS_REVIEW > AI_PROCESSING lets staff re-run the model. Every transition is validated in `services/pods/state.py` and audited.

- **Upload.** POST /trips/{id}/pods with an *Idempotency-Key*; the server validates the image by decoding it (type, size), stores it, writes pod + upload_job + audit in one transaction, returns 201 (or 200 with the original resource on a duplicate key), then processes asynchronously.

- **Review.** POST /pods/{id}/review (APPROVE | REJECT | RETAKE). Corrections are validated per field, require a reason, and are stored beside the untouched AI values; every decision writes audit rows. Only NEEDS_REVIEW can be reviewed; a rejected POD stays in history.
- **Billing.** Eligible = trip not cancelled, latest POD VERIFIED, LR / e-way bill / client / delivery date / freight present. POST /billing/exports writes a CSV, an export record with its trip list and marks trips EXPORTED. Re-exporting an exported trip is refused unless *reexport=true* with a reason. INVOICED can only be recorded manually with an external invoice reference and invoice date. Each CSV row carries evidence references: pod_reference (exact attempt), pod_image_reference (PODIMG-id), pod_verified_date, pod_exceptions and pod_evidence_url (staff login); no storage paths or public image URLs.
- **Client evidence sharing.** From the billing queue (*Share PODs*), export history (*Client link*) or *Client POD links*, a billing executive creates a read-only link for one client, scoped to selected trips or a billing cycle, expiring in 1-30 days. The client sees trip reference, LR, delivery date, POD status and reference and the verified image only; never driver, phone, freight, GPS or other clients' trips. Revocation or expiry returns 410; creation, opens, image views, denied attempts and revocation are audited. A client token can never resolve as a driver upload token (separate table and URL space).
- **Hired driver.** Billing creates a 256-bit random token (only its SHA-256 is stored), scoped to one trip, expiring (default 24 h). The /upload/{token} page needs no account, can only write a POD (and its original) for that one trip, and is revocable; opens and uploads are audited. A mock provider writes a delivery-log row; **no SMS is sent**.

8. AI document processing architecture

After upload a background task reads the stored image, runs a cheap blank-image pre-check, and calls the provider with a system prompt that forbids guessing, a tool whose JSON Schema requires every field (null when unreadable; *tool_choice=auto* because the model rejects forced choice, so the prompt mandates the call and a missing call is an *invalid_output* error), and **no expected trip values** (so the model reads rather than confirms). Output is validated by a strict Pydantic model (`extra=forbid`, confidences in [0,1], ISO dates) — failure is an *invalid_output* error. Extracted: LR number, consignee, delivered quantity + unit, delivery date, signature visible, stamp visible, shortage / damage flags, verbatim remarks, readability and observations. Every run stores provider, model id, prompt version (pod-extract-1.0), schema version (1.0), duration, field confidences, validation checks and any error.

Failure	Handling
Timeout / network / rate limit / 5xx	Typed AIProviderError; POD kept; routed to NEEDS_REVIEW with reason AI_ERROR; staff can *Re-run AI extraction*.
Invalid structured output / unsupported or unreadable image	Same, or retake when the image is blank/unreadable and retakes remain.
Missing API key / provider disabled	Not-configured error recorded; every POD goes to a human (manual mode).
Any exception during processing	Caught; extraction row records the failure; the document is never lost.

Provider verification status: the adapter is contract-tested against a stubbed HTTP layer (request shape, success parsing, 400/401/429/5xx/timeout/network and invalid-output mappings) **and was exercised live** with model *claude-sonnet-5-5* on three synthetic documents (credit use was deliberately limited). A clean signed+stamped POD was read correctly (LR, consignee, 600 bags, date; confidences 0.95-0.97) and auto-verified through the real API path in 4.7 s; a shortage document returned 594 bags, 'Short by 6 bags' and shortage_detected=true. **This is three samples, not an accuracy measurement**: no accuracy, cost or load figures are claimed, and blurred, wrong-LR, missing-stamp and damage photos were not run live (covered by mock tests). Seeded demo extractions are labelled *seed-data*.

9. Confidence handling and human review

Rule (implemented in services/ai/validation.py)	Outcome
Critical field (LR, consignee, date) confidence below threshold (default 0.85)	NEEDS_REVIEW (LOW_CONFIDENCE)
Shortage remark, damage remark, LR mismatch, consignee mismatch, quantity mismatch (only if expected quantity exists), implausible date	NEEDS_REVIEW (high / medium priority)
Missing LR/consignee/date, stamp or signature not visible, unreadable image	RETAKE_REQUESTED while retakes remain (cap 2), else NEEDS_REVIEW
Provider error	NEEDS_REVIEW (AI_ERROR)
All checks pass	VERIFIED by automated checks (see A2); high confidence never overrides a failed check

The review screen (shared by the AI verification screen) shows the original image with zoom/rotate, Expected vs AI-read per field, field confidence, check results and highlights for matches, mismatches, missing and low-confidence values. Signature and stamp are shown as *visible/not visible* with an explicit notice that **RouteProof does not authenticate them**. Reviewers edit a field, give a reason, then approve, reject (reason required) or request a retake (disabled at the cap). Both the AI original and the corrected value stay visible, in the review history and in the audit log.

10. Offline-first architecture

- **Capture:** file input with *capture=environment* opens the phone camera (also works on basic Android); GPS requested with timeout and permission handling; capture UUID, trip id, client timestamp, coordinates/accuracy stored.
- **Compression and originals:** Canvas re-encode to JPEG, stepping quality then dimension until <300 KB. The original and compressed blobs are separate IndexedDB fields. The compressed copy is uploaded first; after it is acknowledged the original is sent in a lower-priority pass (good connection only, 180 s timeout, up to 5 attempts) and released only when acknowledged. A failed original never affects the accepted POD and stays on the phone.
- **Queue:** record is persisted *before* any network call. Statuses: PENDING (shown as 'waiting for network' when offline), UPLOADING, UPLOADED, FAILED, RETRY_REQUIRED (after 3 transient failures; manual retry button). A single-flight flush runs on the browser *online* event, every 20 s, on tab focus, at start-up and via the retry button.
- **Idempotency:** the capture UUID is the Idempotency-Key on every attempt; the server enforces a unique key and returns the original POD on duplicates. Keys cannot be reused across trips.
- **Acknowledgement:** the compressed blob is released after its upload is acknowledged, the original only after its own upload is acknowledged. Permanent 4xx errors become FAILED with the server message instead of retrying forever.
- **PWA:** manifest + production service worker caching the app shell; Background Sync merely wakes open pages (uploads need the session cookie or link token), so manual and foreground retry are the guaranteed path.

11. UI/UX design decisions and key screens

Two deliberately different interaction models on one design system (forest #183B36, teal #2D7768, canvas #F3F5F2, amber #D69B37, Inter). **Driver:** one giant camera button, large cards showing the LR number, icon-first status (green = verified, amber = retake/review, red only for real errors), Hindi/Marathi/English toggle with browser speech synthesis where a voice exists and on-screen text otherwise, persistent online/offline banner. **Executive:** sidebar workspace, compact KPI cards, dense tables with right-aligned tabular numbers, filters on every list, split-pane document review. Every list has skeleton, empty and error-with-retry states; destructive or irreversible actions use confirmation dialogs; toasts report outcomes.

Screen	Route	Notes
Operations dashboard	/dashboard	KPIs, aging chart, client stacked bars, branch table, recent activity; filters date/client/branch; labelled *Seeded demo data* vs *Live* from DATA_MODE; every number from /dashboard/*
Trips + detail	/trips, /trips/{id}	search, 6 filters, pagination, CSV import (dry-run, duplicate + row-error report); POD attempts, audit history, billing history, hired-driver link
POD capture (driver)	/driver, /driver/capture/{id}, /driver/queue	offline indicator, preview/retake, GPS chip, state tracker, queue with retry
AI verification + review	/pods/{id}, /reviews	split view, confidence, validation, corrections with reasons, approve/reject/retake, priority queue
Billing queue	/billing	verified trips only, client/cycle/date/search filters, selection, CSV, export history, explicit re-export
Missing PODs	/operations	actionable overdue list, elapsed days, freight held back, pending reviews
Hired driver	/upload/{token}	mobile landing, no login, same capture engine
Client links / client view	/client-links, /view/{token}	create, list, revoke; read-only verified POD evidence for one client

12. Security and data protection

- **AuthN/Z:** Argon2 hashes; JWT in an HttpOnly, SameSite cookie (not readable by page scripts, so an XSS bug cannot exfiltrate it) with a session-bound CSRF token required on every state-changing request; 60-min idle / 12-h absolute expiry; production refuses non-secure cookies. 401/403 enforced in FastAPI dependencies. Drivers are limited to assigned trips by a resource-level check that returns 404 (not 403) for others' trips, PODs and images; drivers receive no freight/billing fields. Billing-only endpoints return 403 to drivers (tested).
- **Input:** Pydantic validation, CSV size/row limits, image validated by decoding (not by Content-Type), 5 MB cap, CSV formula-injection neutralisation, safe error envelope without stack traces.
- **Abuse:** in-memory rate limits on login and link endpoints (per instance); unknown-user logins cost the same hash verification; CORS restricted to configured origins; security headers on the frontend.
- **Secrets:** environment variables only; production start-up fails with a clear message when required secrets are missing; .env is git-ignored; the browser never sees the AI key.
- **Audit:** state change and audit row commit in one transaction (uploads, AI runs, corrections, approvals, exports, link creation/open/upload, login/logout).
- **Privacy (DPDP-aware, not a compliance claim):** data minimisation (no Aadhaar or unrelated identifiers; phone numbers only masked in logs; fictional demo data); POD images and consignee details treated as sensitive; images private and served only to authorised users; **Retention:** verified images of invoiced trips 2,920 days, superseded attempts 365 days, access logs 365 days (configurable); `python -m app.services.retention` is a dry run, *--apply* purges, *--erase-pod* with a reason handles an approved erasure request. **Legal hold:** images of a trip not yet invoiced or with an open POD are never purged; erasing one needs *--override-hold*, which is recorded. Purged images answer 410, the record is kept, and every purge writes an **audit** row without the deleted content. Nothing is deleted on a timer. Sending images to a third-party model is a cross-border processing decision for the client's legal team. **A proper legal and security review is required before production use.**

13. Testing strategy and actual results

Layer	Scope	Result (run in the authoring environment)
Backend unit + API (pytest, httpx TestClient, SQLite, mock AI)	routing policy, thresholds, schema strictness, state machine, retake cap, eligibility, CSV escaping, idempotency, auth, RBAC, driver scoping, upload validation, duplicate upload, AI timeout/invalid output/unreadable, review corrections + audit, queue filters, export + duplicate protection + re-export, hired-link and client-link security (scoping, expiry, revocation, token separation, audit), retention/erasure, every-endpoint auth sweep, labelled AI-routing evaluation set (31 cases), dashboard, CSV import, Claude adapter contract	**201 passed, 2 opt-in skipped (1 demo-generator test needs scripts/ beside backend/)** on SQLite and on PostgreSQL 16 (CI runs both)
Frontend (Vitest + Testing Library + fake-indexeddb)	login, dashboard rendering and error state, trip search, verification highlighting, correction-reason rule, retake cap, billing export and re-export, offline queue (retry with same key, ack releases blobs, no concurrent flush, offline skip), i18n and voice fallback, compression policy	**57 passed**
End-to-end (Playwright, Chromium, real browser + real API with deterministic mock AI)	driver capture (<300 KB) > AI > review > correction > approval > billing queue > CSV download > audit; clean POD auto-verified; AI timeout; two retakes then review; authorization; dashboard; hired-driver link journey, client view + revocation, offline capture surviving an app restart, lost-response retry, server timeout, sign-out with unsent photos	**16 Playwright specs**
Static checks	ESLint, `tsc --noEmit` (strict), `next build`	clean / clean / success

Testing found and fixed four real defects: (a) PostgreSQL enforced an insert-order foreign key (upload job before its POD) that SQLite hid; (b) the live model rejects forced tool_choice; and (1) a single-flight lock released before it was assigned when the queue flushed while offline, which would have blocked in-page auto-sync after reconnection; (2) dialogs re-grabbed focus on every keystroke, breaking reason entry. **Verified on PostgreSQL 16:** migration, idempotent seed, JSONB/NUMERIC/UUID column types and the full pytest suite. **Not tested:** the production service worker offline-navigation path, real Android devices, load/performance, and any cloud deployment. The Playwright suite uses SQLite, the mock model and the Next.js dev server by design. GitHub Actions (.github/workflows/ci.yml) runs backend tests on SQLite and PostgreSQL 16, frontend lint/typecheck/tests/build and Playwright on every push and pull request to main.

14. Deployment architecture

Target: frontend on Vercel (or a second Render service) → FastAPI container on Render (`render.yaml`, health check `/health`) → managed PostgreSQL + persistent disk for images (or an S3-compatible private bucket via `STORAGE_PROVIDER=s3`). The container runs `alembic upgrade head`, an idempotent seed, then uvicorn. Render has no India region, so the blueprint uses Singapore; for India residency choose AWS ap-south-1 (Mumbai) storage/database. **Status: configuration and instructions are complete; nothing has been deployed or verified live.** A single instance is assumed (disk storage, in-memory rate limiter); scale-out needs S3 and a shared limiter.

15. Known limitations

- Live extraction was checked on only three synthetic documents; quality on real, messy photographs, latency and cost are unmeasured and the 0.85 threshold is uncalibrated.
- Cloud deployment is pending (see sections 2 and 14); the Render blueprint uses Singapore. The Alembic baseline uses metadata create-all; only incremental revisions are true diffs.
- Sessions are stateless JWT cookies: logout clears them but cannot revoke a copied token before expiry (no denylist). In-memory rate limiting is per instance. Retention runs on demand, not on a timer. AI runs in FastAPI BackgroundTasks with database-backed stale-job recovery, not a durable queue. Tally and SMS are mocked.
- Camera access relies on the OS camera app through a file input (robust on basic Android, but no in-page live preview). PDFs and multi-page PODs are not supported; only JPEG/PNG/WebP.
- Dashboard aggregation loads filtered trips and computes in Python: fine at the stated volume, to be moved to SQL aggregates if volumes grow. (The review queue already filters, orders and paginates in SQL.)
- Hindi/Marathi strings were written by the developer and need native-speaker review; speech depends on installed browser voices.
- Background Sync cannot upload by itself while the app is closed. Real-Android camera, airplane-mode, GPS-denied, voice and production offline-reload checks are still manual.

16. Features deliberately excluded

Real SMS delivery (mock adapter only), live Tally integration (CSV only), automatic invoice generation/sending, e-way bill or GPS-tracking integrations, signature/stamp authentication, payments, a third role, microservices, multi-tenant administration, and push notifications.

17. Future improvements

- Calibrate thresholds from a labelled sample of real PODs; track precision of auto-verification and reviewer-override rate.
- Server-side session revocation list; shared rate limiter; malware scan on uploads.
- SQL-side aggregations and materialised metrics; background worker queue for AI jobs with retries and dead-lettering.
- On-device quality guidance (blur / glare / edge detection) before sending; multi-page capture; PDF support.
- Scheduled retention runs; field-level encryption for sensitive images; region-pinned storage; accessibility audit and native-speaker localisation review.
- Real SMS/WhatsApp provider, Tally import automation, and e-invoicing handled as separate, approved integrations.